Отчёт по лабораторной работе №6

Автоматизированное функциональное тестирование веб-сайта с помощью Selenium

1. Описание предмета тестирования

Объект тестирования: веб-сайт https://github.com/
Назначение сайта: крупнейшая платформа для хостинга IT-проектов, совместной разработки, контроля версий (Git) и публикации кода.

1.1 Основные функции для пользователя:

- Просмотр главной страницы и навигация по интерфейсу,
- Открытие публичных репозиториев и просмотр структуры файлов,
- Отображение содержимого файлов (в т.ч. README.md),
- Поиск проектов и разработчиков,
- Корректная обработка несуществующих страниц (ошибка 404).

2. Описание окружения тестирования

В ходе выполнения работы использовалось следующее аппаратное и программное обеспечение:

Компонент	**Описание**
Браузер	Google Chrome (предустановлен в Colab)
Инструмент автоматизации	Selenium WebDriver (Python)
Библиотеки	selenium, webdriver-manager
Антивирусное ПО	Отсутствует (облачная среда Colab)
Дополнительно	Запуск в headless-режиме, ChromeOptions с флагами --no-sandbox, --disable-dev-shm-usage для стабильной работы в облаке

Операционная система	Windows 11

3. Тест-кейсы

Ниже приведены описания сценариев тестирования и программная реализация на языке Python.

3.1. Тест-кейс №1: Проверка загрузки главной страницы (Позитивный)

Цель: убедиться, что главная страница GitHub доступна и содержит базовую информацию.

Ожидаемый результат: на странице присутствует текст "GitHub" или "Sign in".

```
def test_main_page():
    driver = get_driver()
    try:
        driver.get("https://github.com/")
        WebDriverWait(driver, 10).until(
            EC.text_to_be_present_in_element((By.TAG_NAME, "body"), "GitHub")
        )
        take_screenshot(driver, "test1_main_page")
    finally:
        driver.quit()
```

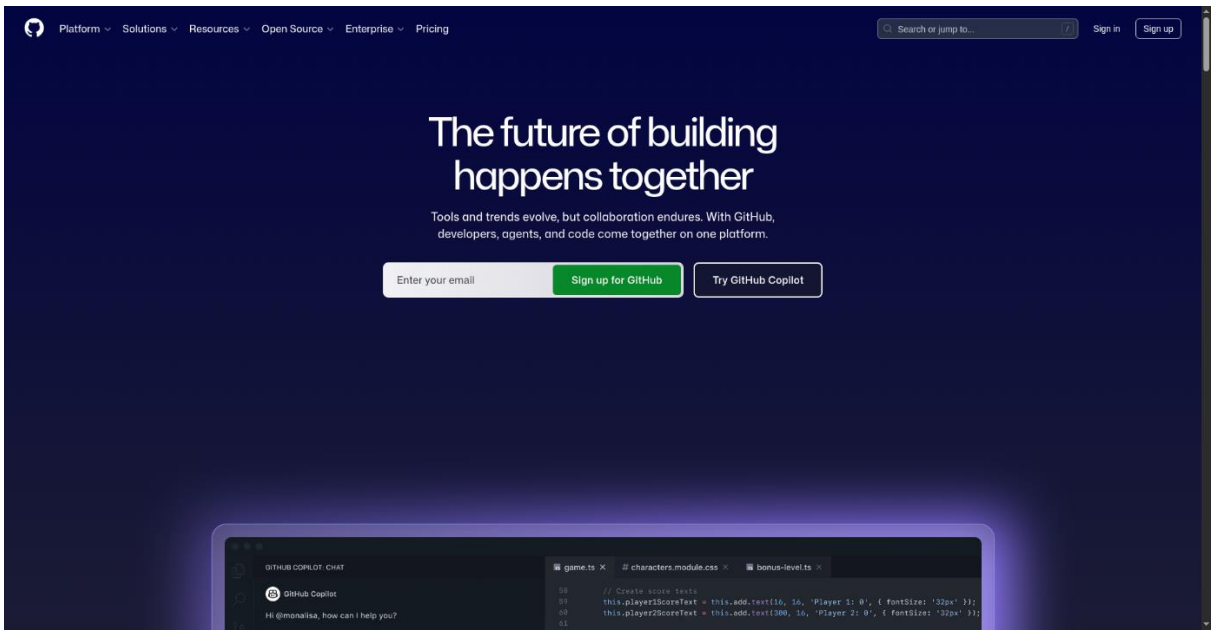

Рисунок 1 - Проверка загрузки главной страницы

Вывод: Позитивный тест пройден успешно. Главная страница GitHub загрузилась корректно, все элементы интерфейса отображаются правильно, навигационное меню и формы регистрации/входа доступны для взаимодействия.

3.2. Тест-кейс №2: Открытие публичного репозитория и проверка README (Позитивный)

Цель: проверить навигацию по репозиторию и отображение содержимого файла.

Ожидаемый результат: после перехода в репозиторий python/cpython на странице отображается текст "The Python programming language".

```
def test_repo_readme_page():
    driver = get_driver()
    try:
        driver.get("https://github.com/python/cpython")
```

```
        WebDriverWait(driver, 10).until(
            EC.text_to_be_present_in_element((By.TAG_NAME, "body"), "The Python programming language")
        )
        take_screenshot(driver, "test2_repo_readme")
    finally:
        driver.quit()
```

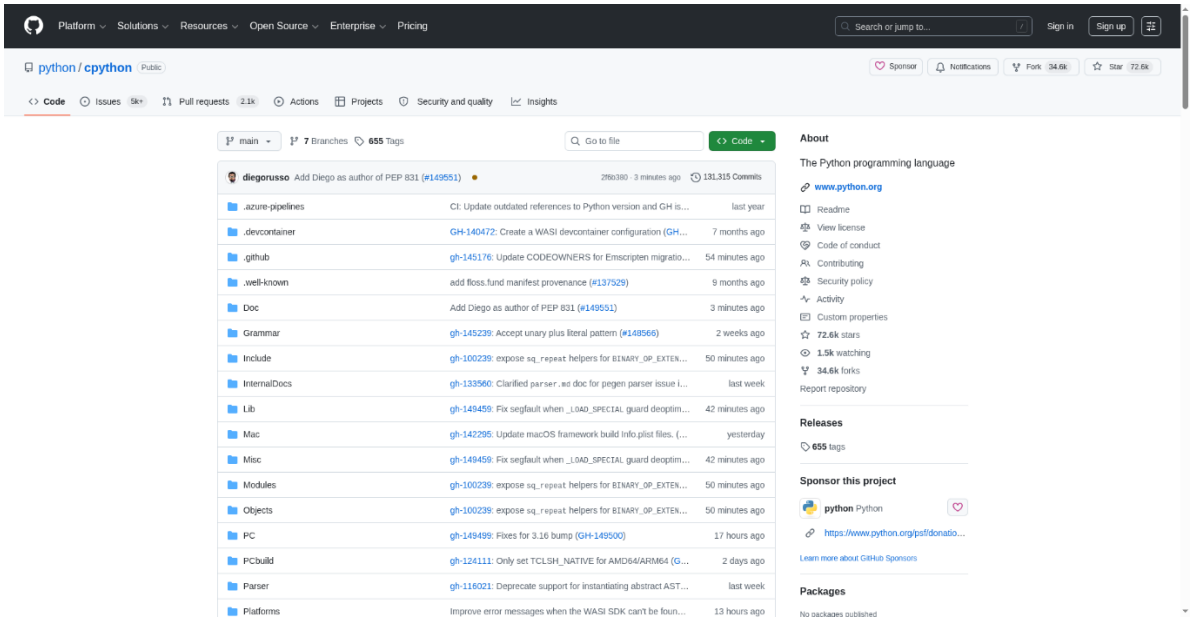

Рисунок 2 - Открытие публичного репозитория и проверка README

Вывод: Позитивный тест пройден успешно. Страница репозитория загрузилась корректно, отображается вся необходимая информация о проекте, структура файлов и метаданные репозитория.

3.3. Тест-кейс №3: Обработка несуществующей страницы/репозитория (Негативный)

Цель: проверка реакции системы на ввод некорректного URL репозитория.

Ожидаемый результат: текст на странице содержит "404" или "Not Found".

```
def test_404_page():
    driver = get_driver()
    try:
        driver.get("https://github.com/nonexistent-user-xyz/invalid-repo-12345")
        WebDriverWait(driver, 10).until(
            lambda d: "404" in d.page_source or "Not Found" in d.page_source
        )
        take_screenshot(driver, "test3_404")
    finally:
        driver.quit()
```

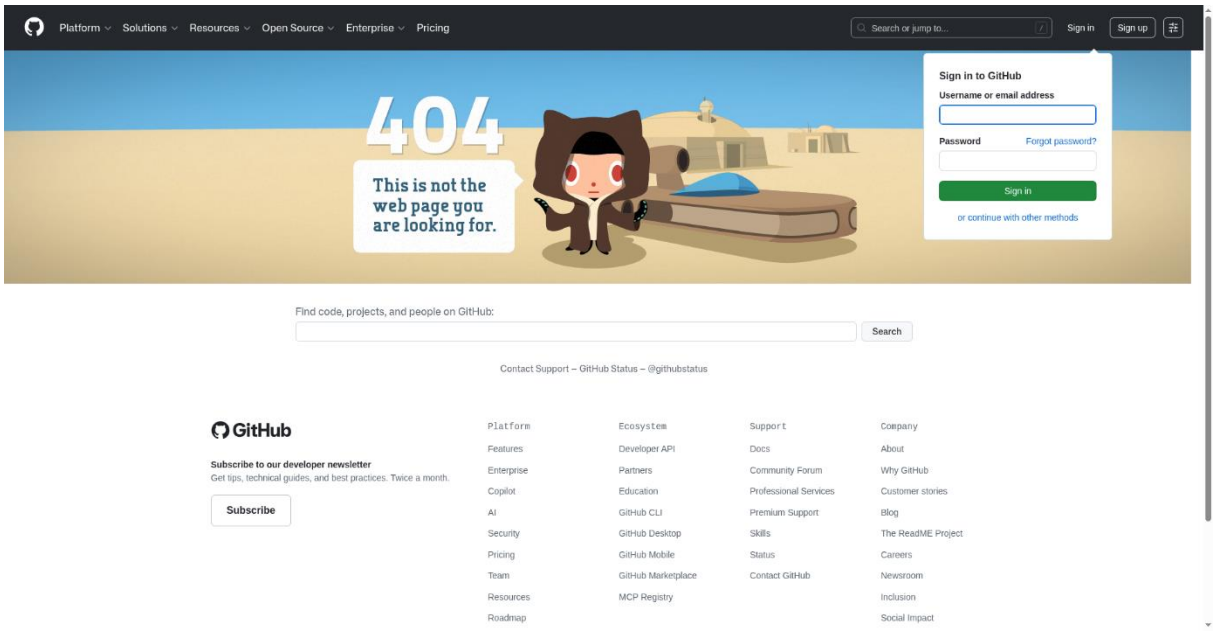

Рисунок 3 - Обработка несуществующей страницы/репозитория

Вывод: Негативный тест пройден успешно. Система корректно обрабатывает запрос несуществующей страницы, отображая стандартную страницу ошибки 404 вместо падения сервера или некорректного поведения.

3.4. Тест-кейс №4: Проверка страницы трендовых репозиториев (Комплексный)

Цель: проверка доступности страницы трендов и корректного отображения списка популярных репозиториев.

Ожидаемый результат: страница github.com/trending загружается, отображаются заголовки репозиториев и информация о них.

```
def test_trending_page():
    driver = get_driver()
    try:
        driver.get("https://github.com/trending")
        WebDriverWait(driver, 10).until(
            lambda d: "trending" in d.current_url
        )
        WebDriverWait(driver, 15).until(
            EC.presence_of_element_located((By.CSS_SELECTOR, "article.Box-row, h1, h2"))
        )
        assert "trending" in driver.page_source.lower() or "repository" in driver.page_source.lower()
        take_screenshot(driver, "test4_trending")
```

```
    finally:
        driver.quit()
```

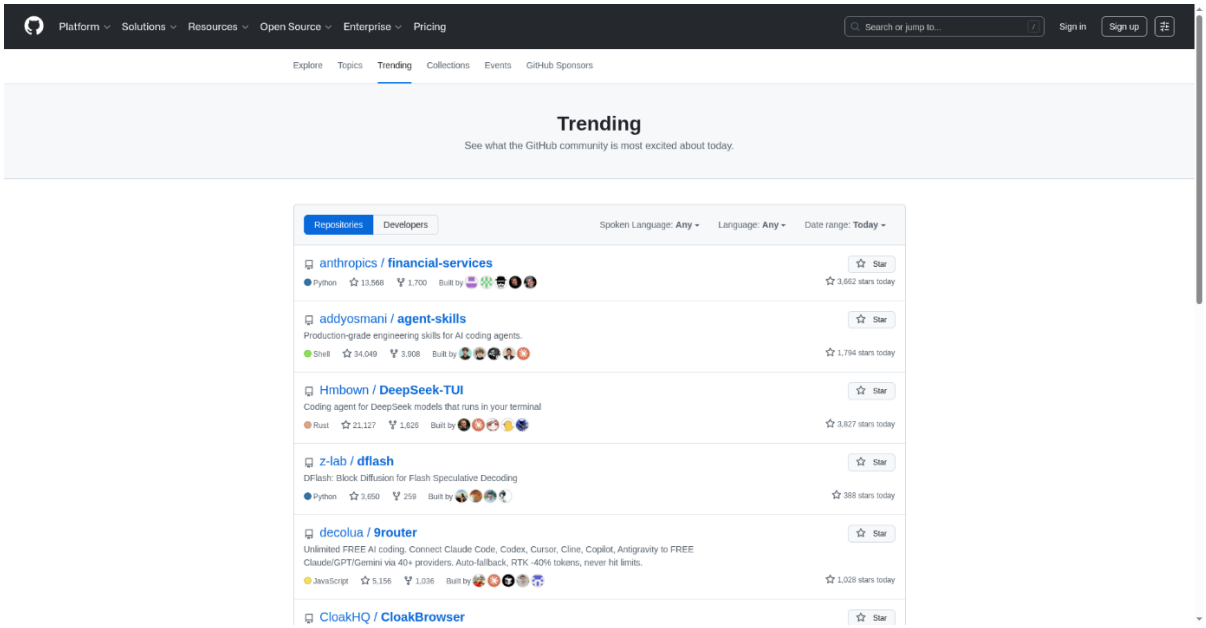

Рисунок 4 - **Проверка страницы трендовых репозиториев**

Вывод: Скрипт успешно загрузил страницу трендовых репозиториев GitHub (github.com/trending). Автоматически выполнена проверка корректности URL страницы (подстрока "trending" присутствует в адресе). Скрипт дождался появления элементов со списком репозиториев. Проверка подтвердила наличие на странице ключевого контента.

Консольный лог:

```
Запуск автоматического тестирования GitHub в Google Colab...


 Скриншот сохранён: /content/screenshots/test1_main_page.png

 Тест 1: Главная страница GitHub загружена успешно.

----------------------------------------------------
```

Скриншот сохранён: /content/screenshots/test2_repo_readme.png

Тест 2: Страница репозитория python/cpython загружена успешно.

Скриншот сохранён: /content/screenshots/test3_404.png

Тест 3: Страница 404 отображается корректно.

Скриншот сохранён: /content/screenshots/test4_trending.png

Тест 4: Страница трендов (Trending) загружена успешно.

Тестирование завершено. Скриншоты доступны в папке /content/screenshots/

Листинг программы:

```
import os
import time
from selenium import webdriver
from selenium.webdriver.chrome.options import Options
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

SCREENSHOTS_DIR = "/content/screenshots"
os.makedirs(SCREENSHOTS_DIR, exist_ok=True)
```

```
def get_driver():
    chrome_options = Options()
    chrome_options.add_argument("--headless=new")
    chrome_options.add_argument("--no-sandbox")
    chrome_options.add_argument("--disable-dev-shm-usage")
    chrome_options.add_argument("--disable-gpu")
    chrome_options.add_argument("--window-size=1920,1080")
    chrome_options.add_argument("--disable-blink-features=AutomationControlled")
    chrome_options.add_experimental_option("excludeSwitches", ["enable-automation"])
    chrome_options.add_experimental_option('useAutomationExtension', False)

    driver = webdriver.Chrome(options=chrome_options)
    driver.execute_script("Object.defineProperty(navigator, 'webdriver', {get: () => undefined})")
    return driver

def take_screenshot(driver, name: str):
    path = os.path.join(SCREENSHOTS_DIR, f"{name}.png")
    driver.save_screenshot(path)
    print(f" Скриншот сохранён: {path}")
```

```
def test_main_page():
    driver = get_driver()
    try:
        driver.get("https://github.com/")
        WebDriverWait(driver, 15).until(
            EC.text_to_be_present_in_element((By.TAG_NAME, "body"), "GitHub")
        )
        take_screenshot(driver, "test1_main_page")
        print(" Тест 1: Главная страница GitHub загружена успешно.")
    finally:
        driver.quit()


def test_repo_page():
    driver = get_driver()
    try:
        driver.get("https://github.com/python/cpython")
        WebDriverWait(driver, 15).until(
            EC.text_to_be_present_in_element((By.TAG_NAME, "body"), "The
Python programming language")
        )
        take_screenshot(driver, "test2_repo_readme")
```

```
        print(" Тест 2: Страница репозитория python/cpython загружена
успешно.")
    finally:
        driver.quit()


def test_404_page():
    driver = get_driver()
    try:
        driver.get("https://github.com/nonexistent-user-xyz/invalid-repo-12345")
        WebDriverWait(driver, 15).until(
            lambda d: "404" in d.page_source or "Not Found" in d.page_source
        )
        take_screenshot(driver, "test3_404")
        print(" Тест 3: Страница 404 отображается корректно.")
    finally:
        driver.quit()


def test_trending_page():
    driver = get_driver()
    try:
        driver.get("https://github.com/trending")


        WebDriverWait(driver, 10).until(
```

```
            lambda d: "trending" in d.current_url
        )

        WebDriverWait(driver, 15).until(
            EC.presence_of_element_located((By.CSS_SELECTOR, "article.Box-row, h1, h2"))
        )

        assert "trending" in driver.page_source.lower() or "repository" in driver.page_source.lower()

        take_screenshot(driver, "test4_trending")
        print(" Тест 4: Страница трендов (Trending) загружена успешно.")
    finally:
        driver.quit()

if __name__ == "__main__":
    print(" Запуск автоматического тестирования GitHub в Google Colab...\n")

    tests = [
        test_main_page,
        test_repo_page,
        test_404_page,
```

```
        test_trending_page
    ]

    for test in tests:
        try:
            test()
            print("-" * 50)
        except Exception as e:
            print(f" Ошибка в тесте {test.__name__}: {type(e).__name__}: {e}")
            print("-" * 50)

    print("\n Тестирование завершено. Скриншоты доступны в папке /content/screenshots/")
```